Hibernate

- ***Hibernate*** - It is an ORM (Object relational Mapping) tool, framework for Java that simplifies database interactions by mapping Java classes to database tables. Gives you CRUD operations and complex queries without extensive SQL code.
  - used in data layer of applications
  - implements JPA
  - execute native SQL queries

  - **Automatic SQL Generation –** Hibernate creates SQL for common tasks (INSERT, SELECT, UPDATE, DELETE) so you don't have to write raw SQL manually. Developers interact with methods like save(), update(), and delete().
  - **Database Independence** - Hibernate works with different databases (MySQL, PostgreSQL, Oracle) so you can switch between databases with minimal changes (it has vendor-neutral abstraction layer)
  - **Eliminates Boilerplate JDBC Code** - Hibernate replaces manual JDBC tasks like managing connections, writing SQL, and mapping results. With HQL (Hibernate Query Language) and its ORM features, you don't have to do this manually, which makes code cleaner, easier to maintain, and faster to develop.

Maven

- ***Maven*** – is a build tool in managing and getting dependencies of a java project.
  - Central Repository, Remote repository, Local repository - located locally/machine

Sample pom.xml

```
<project xmlns:>
  <groupId>com.sample</groupId>
  <artifactId>my_project</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>My Project</name>
  <packaging>war</packaging>

  <dependencies>
      <dependency>
          <groupId>org.springframework</groupId>
          <artifactId>spring-webmvc</artifactId>
          <version>4.3.8.RELEASE</version>
      </dependency>
  </dependencies>
</project
```

Spring Framework

Spring Framework-

- an application framework and inversion of control container for Java used for building scalable applications.
- It is light-weighted, loosely coupled and is created to reduce the complexity of enterprise application development.
- It provides features such as Dependency Injection (DI) to manage object creation and injection.
- It has a variety of modules which help simplify Java development and these include:
  - Spring MVC for web applications,
  - Spring Data for data access
  - Spring Boot for creating stand-alone, production-ready applications with minimal configuration.

- ***How Does Spring Framework Work?*** The Spring Framework primarily operates based on two core principles: Inversion of Control (IoC) and Dependency Injection (DI).
  - ***Inversion of Control (IoC)-*** *is a design principle where the control flow of the program is inverted. In traditional programming, the application is responsible for creating and managing objects. With IoC, the control flow of a program is given to the framework, which manages the creation, configuration, and lifecycle of objects.*
    - *Leads to more modular and flexible code, minimizes the amount of code in an application and promotes loose coupling.*

      - *<u>Tight Coupling</u> – is when a class is dependent on another class's object. – this means that the dependent class is responsible of creating its dependencies.*
      - *<u>Loose Coupling</u> –*
        - *is when you remove the dependency of an object on a class.*
        - *(We use @Autowired annotation which means that Spring takes control of creating the instance and injects the dependency)*
        - *is approached by creating an interface and a setter & getter method, or by using a constructor which takes the interface object.*
    - ***Types of IoC Container***
      - *BeanFactory container –*
        - is like a factory class that contains a collection of beans.
        - Uses Lazy initialization.
        - Suitable for lightweight applications where advanced features are not required.
        - Example: XmlBeanFactory *(though it's now deprecated in favor of ApplicationContext).*
      - *ApplicationContext Container –*
        - built on top of BeanFactory (A more advanced version of BeanFactory).
        - Uses Eager initialization.
        - Provides extra functionalities like AOP and web application context.
        - Example*: ClassPathXmlApplicationContext, WebApplicationContext (for web applications)*

  - ***Dependency Injection (DI)***:
    - is a pattern used to implement IoC. It is merely one concrete example of Inversion of Control.
    - involves injecting dependencies into a class rather than having the class create its dependencies.
    - simply removing hard coded wiring in the app and using a framework to inject dependency resources when they needed.
    - decouples the creation of objects from their usage, promoting loose coupling.
    - achieved through constructor injection, setter injection, or field injection. *Types of Injection/inject Beans*
      - *<u>Constructor Injection</u> – annotation is used on a constructor, use for mandatory dependencies, work better in many properties. No partial injection and will create a new instance if any modification is done. Constructor injection allows injecting values to immutable fields and makes testing easier.*
      - *<u>Setter Injection</u> – annotation is used on the setter method, use for optional dependencies, works better for few properties. Partial injection and will not create instance if any modification is done.*
      - *<u>Field Injection</u> - annotation is directly used on properties. This is how it eliminates the need for getters and setters.*

Advantage of Spring (SFRECI)

- ***Simplified Application development due to Dependency Injection and Inversion of Control***.
- ***Flexible, modular and backwards compatible***. ***Flexibility***: The modular nature of Spring means that developers can use only the components they need. This flexibility allows for a customized setup tailored to specific project requirements, whether it's a simple web application or a complex enterprise solution.
- ***Reduces Boilerplate Code***: Spring eliminates much of the repetitive and cumbersome code allowing developers to focus on business logic. One of Spring's main advantages is its ability to reduce boilerplate code. Through its DI and AOP features, Spring eliminates much of the repetitive and cumbersome code typically associated with enterprise application development.
- ***Enhanced Testability***: Spring's DI makes it easy to inject mock dependencies, facilitating unit and integration testing. This leads to higher code quality and easier identification and resolution of bugs.
- ***Consistency***: Spring provides consistent programming and configuration models across different types of applications. Whether we develop a web application, a microservice, or a batch job, Spring offers a unified approach, reducing the learning curve and improving developer productivity.
- ***Improved Productivity***: Spring Boot enables developers to create stand-alone, production-ready applications quickly, with minimal configuration. It provides convention-over-configuration approaches and embedded servers.

  Tools like Spring Boot, part of the Spring ecosystem, significantly enhance developer productivity by providing convention-over-configuration approaches and embedded servers. Spring Boot enables developers to create production-ready applications quickly, with minimal configuration.

Features of Spring - Spring offers a rich set of features that cater to various aspects of application development:

1. ***Dependency Injection (DI) -*** Spring Context : Facilitates the management of dependencies and promotes loose coupling.
2. ***Aspect-Oriented Programming (AOP):*** Allows separation of cross-cutting concerns from business logic.
3. ***Transaction Management***: Provides a consistent programming model for transaction management, supporting both declarative and programmatic transaction management.

4. ***Spring MVC*** – for building web applications that uses a powerful Model-View-Controller (MVC) web framework.
5. ***Spring Boot*** – for creating standalone, production ready application with minimal configuration and embedded server.

6. ~~***Spring Web Module –*** for creating web applications.~~
7. ***Spring Data*** – for simplified data access and manipulation.
8. ***Spring DAO*** – for database operations using DAO pattern
9. ***Spring JDBC*** – for JDBC and DataSource support.

10. ~~***Spring ORM*** – for ORM tools support such as Hibernate~~
11. ***Spring Security*** – for authentication and authorization mechanisms for securing applications.
12. ***Spring Batch*** –for batch or job processing support, including transaction management.

POJO – plain old java objec.
JavaBeans – simple objects with only getters and setters.
Spring Container – is a container of beans.
Spring Beans - are objects that forms the backbone of spring application. Object that is instantiated, assembled and managed by Spring IOC.
Bean – essentially class Annotated by @Component.

Spring Bean Lifecycle:

The lifecycle of a bean is managed by the Spring container (BeanFactory) which is responsible for creating, configuring, and destroying beans according to the defined bean scope (singleton, prototype, etc.).

- Instantiated -> Spring bean needs to be instantiated, based on Java or XML bean definition.
- Utilized ->
- Destroyed -> when the bean is no longer required, it will be removed from the IoC container.

Scope of Bean – it controls/defines how long your beans live

1. Singleton – This scopes the bean definition to a single instance per Spring IOC container (default).
2. Prototype – This scopes the bean definition to have multiple number of object instances.

   -- Following is for web aware spring projects --

3. Request - a new bean will be created for each HTTP request. meant to be used in web application.
4. Session - a new bean will be created for each HTTP session.
5. Global Session - used to create global session beans. Global HTTP-session.

In main method of a Spring Application you need to declare BeanFactory or ApplicationContext
BeanFactory – small project, extends to ApplicationContext
ApplicationContext – big web application .

Ways of configuring Spring-based applications

1. ***<u>Xml Based Configuration – simple to use</u>***

```
public class Application {
   public static void main(String args[]) {
      ApplicationContext context = new ClassPathXmlApplicationContext(applicationContext.xml”);
      // where spring.xml is a an xml file that declares all the beans in the application.
   }
}
```

```
<?xml ?>
<beans>
  <bean id="vehicle" class="com.navin.Telusko.bike">
</beans>
```

```
public class Bike {
  ....
}
```

2. ***Annotation Based Configuration***

@Component - same with the bean in xml based configuration

```
public class Application {

   public static void main(String args[]) {
      ApplicationContext context = new ClassPathXmlApplicationContext(applicationContext.xml”);
      // where spring.xml is a an xml file that declares all the beans in the application.
   }

}
```

```
<?xml ?>
<beans>
 <context:component-scan base-package="com.myproject"></context:component-scan>
  <!-- <bean id="vehicle" class="com.navin.Telusko.bike"> -->  commented out
</beans>
```

```
@Component
public class Bike {
  ....
}
```

Java Configuration in Spring Approach *– no applicationContext .xml, applicationContext.xml replaced by @Configuration*

```
public class Application {
   public static void main(String args[]) {
      ApplicationContext context = new AnnotationConfigurationApplicationContext(AppConfig.class);
   }}
```

@Bean annortation only applies to method

```
@Configuration
@ComponentScan({"com.myproject"})
public class AppConfig {

  @Bean(name = "speakerService")
  @Scope(value=BeanDefinitddddddddddddddfrion.Scope_Singleton)
  public SpeakerService getSpeakerService() {
     return SpeakerServiceImpl();
  }
}
public class SpeakerService{

  @Bean(name = "speakerRepository")
  @Scope(value=BeanDefinition.Scope_Singleton)
  public SpeakerRepository getSpeakerRepository() {
     return HibernateSpeakerRepositoryImpl ();
  }
```

```
}

public class SpeakerRepository {}
```

Remove @beans annotation to change to @Service and @Repository. It removes the need of bean configuration

```
@Configuration
@ComponentScan({"com.myproject"})
public class AppConfig {

  public SpeakerService getSpeakerService () {
     return SpeakerServiceImpl ();
  }
}

@Service("speakerService")
@Scope (value=BeanDefinition.Scope_Singleton)
public class SpeakerService {

  public SpeakerRepository getSpeakerRepository() {
     return HibernateSpeakerRepositoryImpl ();
  }
}

@Service("speakerRepository")
@Scope (value=BeanDefinition.Scope_Singleton)
public interface SpeakerRepository extends JpaRepository <Speaker, Id>{ //or CrudRepository

            @Query(name="getSpakersByAge", value="from Speaker where speaker.age > 8 ")
            List<Speaker> getSpakersByAge();
}

// using @Controller
@Controller
@RequestMapping("/myPeople")
public class PeopleController {

            @GetMapping //@PostMapping, @PutMapping
            public String myMethod(@RequestParam(value="myId", required=false) Integer myId, Model model) {
                        List<Person> people = this.peopleRepository.FindAll();
                        model.setAttribute("variableVisibleToTemplate", people);
           // people stored to  variableVisibleToTemplate which nameOfHtmlPag.html can be access
                        return "nameOfHtmlPage";
            }
}
// using @RestController
@RestController
@RequestMapping("/myPeople")
public class PeopleController {
```

```
@GetMapping //@PostMapping, @PutMapping, @DeleteMapping
public List<People> myMethod(@RequestParam(value="myId", required=false) Integer myId) {
        List<Person> people = this.peopleRepository.FindAll();
        return people; //return the people variable and omit model
}}
```

Multiple Spring Configuration Files in One Project is recommended to increase maintainability and modularity. You can load multiple Java-based configuration files:

@Configuration @Import({MainConfig.class, SchedulerConfig.class}) public class AppConfig {	load one XML file that will contain all other configs ApplicationContext context = new ClassPathXmlApplicationContext("spring-all.xml"); And inside this XML file you'll have: <import resource="main.xml"/> <import resource="scheduler.xml"/>

Ways of configuring Spring-based applications

1. XML Configuration

- This is the traditional method where you define your beans and other configurations in an XML file.

Xml Copy

```xml
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="myBean" class="com.example.MyBean" />

</beans>
```

2. Java-based Configuration

- With this approach, you use Java classes annotated with `@Configuration` to define beans.

Java Copy

```java
@Configuration
public class AppConfig {

    @Bean
    public MyBean myBean() {
        return new MyBean();
    }
}
```

- This is more type-safe and allows for richer IDE support.

3. Annotation-based Configuration

- You can also use annotations to manage your Spring components and configurations.

Java Copy

```java
@Component
public class MyBean {
    @Autowired
    private AnotherBean anotherBean;
}
```

- Annotations like `@Component`, `@Autowired`, and `@Service` simplify the setup.

4. Properties File

- You can externalize your configuration using `.properties` or `.yml` files.

Properties Copy

```properties
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=password
```

Each of these methods can be used individually or in combination, providing flexible ways to configure and manage your Spring applications. Need more information on a specific type? [1] [2] [3] [4]

[1] github.com [2] github.com [3] github.com [4] github.com

Design Patterns in the Spring Framework

1. ***Singleton pattern* -** ensures only one instance of an object exists per application. It is useful when managing shared resources or providing cross-cutting services, such as logging and caching.

2. ***Prototype Pattern*** - ensures that a new bean instance is created every time it is requested, unlike the singleton scope which reuses the same instance. This is useful when object creation is costly and cloning an existing object provides better performance and flexibility.

3. ***Proxy Pattern* –** a pattern that allows one object (the proxy) to control access to another object (the subject or service). Spring extensively uses this pattern in Aspect-Oriented Programming (AOP) to add functionality like logging, security, and transaction management without modifying the original object.
   - **Benefits:** *Cross-cutting Concerns: Cleanly separates concerns like logging, transaction management, and security.*

4. ***Factory Method pattern* -** defines an interface for creating an object but allows subclasses to alter the type of objects that will be created.

5. ***Template Pattern* -** defines the skeleton of an algorithm in a base class while allowing subclasses to customize specific steps. Spring uses this in JdbcTemplate and RestTemplate, it simplifies repetitive tasks such as database operations and HTTP requests.

Spring MVC

Spring MVC –

- provides a decoupled way of developing web applications.
- follows the Model-View-Controller design pattern. (architectural pattern that separates logic components model view controller.)
- implements spring framework basic core features like Inversion of Control, Dependency Injection.

Model – (The data layer)

- data that is being transferred between the View and Controller components**.**
- manages the application's data, logic, and rules.
- interacts with the database and handles operations like retrieval, insertion, and updates.

View – (The presentation layer)

- visual display/ UI components that user interacts with.
- retrieves data from the model and displays it to the user but doesn't handle business logic.

Controller – (The intermediary)

- wires the view with the model.
- act as an interface between Model and View components to process all the business logic and incoming requests, manipulate data using the Model component and interact with the Views to render the final output.

Differentiate @Component, @Repository and @Service and @Controller?

Typically a web application is developed in layers like the :

- **Controller @Controller** - initial point of client communication, Maps request to methods and execute it.
- **Business @Service** - where the actual code or logic of the application is written.
- **DAO @Repository** - where the database connections and interaction happens.

In such an architecture web application, **@Component** can be used in any of the layers. @Component – tells the spring that you would like to create an object or bean of this class and store it in Spring container.

Ways in Handling exception in Spring MVC (Controller Based, Global Exception Handler and HandlerExceptionResolver)

Spring Boot

- extension of the Spring Framework that simplifies the setup and development of stand-alone, production grade Spring based applications.
- follows "Convention over Configuration," because it provides sensible default setups so that developers can get started quickly with minimal configuration.
- provides a streamlined, opinionated way to jump-start development quickly.

Benefits:

- **Auto-Configuration**: Spring Boot pre-configures your application, reducing boilerplate code, so you can have a Spring application up and running with minimal effort/smallest amount of code.
    - Absolutely no code generation and no requirement for XML configuration
    - Provide opinionated 'starter' dependencies to simplify your build configuration
- **Standalone Applications**: You can create standalone applications without the need for a separate web server. It has embedded server like Tomcat or Jetty.
- **Production-Ready**: It provides features such as metrics, health checks, and application management out-of-the-box, making it easier to get your application ready for production
- **Microservices Support**: Spring Boot is designed for building microservices, with easy integration with other Spring projects like Spring Cloud.

"Streamlined" means making a process more efficient and straightforward by removing unnecessary steps or complexity.
"Opinionated" means the framework makes assumptions and provides default configurations to help developers get started quickly, guiding them towards best practices.

@SpringBootApplication – used to launch up the entire application. Act as a:

- SpringBootConfiguration(@Configuration in a Spring Application.),
- @EnableAutoConfiguration (auto-configures the classes available in the classpath),
- @ComponentScan(classes available under a package).

@SpringBootTest - Spring Boot provides a special annotation to write integration testing. Notice if we use JUnit 4, we must decorate the test class with @RunWith(SpringRunner.class).

```
@Configuration
@EnableAutoConfiguration(exclude = {DataSourceAutoConfiguration.class})
@ComponentScan(basePackages = "com.example.custom")
public class MyApplication {
   public static void main(String[] args) {
      SpringApplication.run(MyApplication.class, args);
   }
}
```

Sources of External Configuration?

- **Command-line properties**: (these are command line arguments). When you start a Spring Boot app, you can pass in options using -- (double hyphen), like --server.port=8080. Spring Boot automatically turns these options into properties and makes them part of the app's environment settings.
- **Application properties -** are those loaded from the application.properties.
- **Profile-specific properties -** are loaded from the application-{profile}.properties file or its YAML counterpart.

REST (Representational State Transfer)

REST- Web standard that http protocol used in data communication. Where every component is a resource and resource can be accessed using Http standard methods.

POST	create a new resource	**@PostMapping("/people /addPerson")** public void addPerson(@RequestBody Person person) {}

GET	for getting/retrieve a resource	**@GetMapping("/people /{id}")**
		public Person getPerson(@PathVariable(value = "id") Long id) {
		
		@GetMapping("/people /search?")
		public Person getPerson(@RequestParam(value="id", required=false) Long id) {}
		
		@GetMapping("/people /")
		public List< Person > getPeople() {

PUT	create a new or update an existing resource	**@PutMapping("/people /{id}")** public void updatePerson (@PathVariable(value = "id") Long id, @Valid @RequestBody Person person)
DELETE	removes a resource	**@DeleteMapping("/people/{id}")** public void deletePerson (@PathVariable(value = "id") Long id)
OPTIONS	get the supported operations on a resource.	

Annotations

Java Annotation - is a tag that represents the metadata i.e. attached with class, interface, methods or fields to indicate some additional information which can be used by java compiler and JVM.

@Controller – for controller classes in Spring MVC project.
@Service – for service classes.
@RequestMapping –is used to map web requests to Spring Controller methods.
@Autowired – for autowiring dependencies in spring. It tells the spring to look for an object in the spring container and inject it to the class. AutoWiring modes. (no, byType, byName and constructor).
@Qualifier – with @Autowired annotation to avoid confusion when multiple instances of bean type is present.
@Scope – for configuring scope of the spring bean.

@Configuration, @ComponentScan and @Bean – for java based configurations.

@ResponseBody – for sending Object as response, usually for sending XML or JSON data as response.
@PathVariable – for mapping dynamic values from the URI to handler method arguments.

AspectJ annotations for configuring aspects and advices, @Aspect, @Before, @After, @Around, @Pointcut etc.

Spring Boot Application

```
@SpringBootApplication(exclude = {RabbitAutoConfiguration.class})
@ComponentScan(basePackages = {"com.heb.rfms", "com.jaf.security"})
@EnableTransactionManagement  //use for @Transactional – rollbacks changes in db if something happened
like exception
@EnableJpaAuditing
@EnableJpaRepositories(
```

```
        entityManagerFactoryRef = "primaryEntityManagerFactory",
        transactionManagerRef = "primaryTransactionManager", basePackages = {
"com.heb.assortment.repository.asrtnt",
                "com.heb.assortment.dectree.repository", "com.heb.assortment.scenariorule.repository" })
@PropertySources(value = {
        @PropertySource("classpath:rfms.properties"),
        @PropertySource("classpath:rfmssqls.properties"),
})
@Slf4j
@EnableBatchProcessing
@EnableScheduling
public class AppConfig extends SpringBootServletInitializer {}
```

```
@Configuration
@EnableBatchProcessing
@EnableScheduling
public class BatchConfiguration extends DefaultBatchConfigurer {}
```

```
@Configuration

public class SchedulingConfig implements SchedulingConfigurer {}
```

JDBC

JDBC – use for interacting with a database using pure SQL. It has no notion of objects or hierarchies and it's up to you to translate/map a result set into Java objects. It allows you to do more things with the Database directly, but it requires more attention.

JDBC Interview Questions

Q1. What is JDBC Driver?

There are 4 types of JDBC drivers:

1. JDBC-ODBC bridge driver
2. Native-API driver (partially java driver)
3. Network Protocol driver (fully java driver)
4. Thin driver (fully java driver)

JPA (Java Persistence API) is a specification that simplifies database interactions in Java using Object-Relational Mapping (ORM). It allows developers to work with Java objects instead of writing complex SQL queries by managing persistence through entities and transactions. Common implementations include Hibernate, which extends JPA with additional features.

JDBC Template Sample Code

```
@Autowired
private JdbcTemplate primaryJdbcTemplate;

private final Log LOG = LogFactory.getLog(AssortmentDaoImpl.class);
private final String CALL_FNCL_FCST = "call FNCL_FCST( :assortmentId, :"
                + "scenarioId, :groupTraitCode, :traitCode, :operator, :traitValueCode, "
```

```
                + ":metricField, :topDownFlag, :groupByType, :filterByType);";

@Override
public List<ConsmUntItmCustomDto> getConsmUntID(int assortmentSessionId) {
        String query = "SELECT I.LGCY_ID AS LGCY_ID, C.CONSM_UNT_ID AS CONSM_UNT_ID,"
                + " C.WHSE_ITM_ID AS WHSE_ITM_ID FROM CONSM_UNT_ITM_XREF C,"
                + " ITM I WHERE C.SSON_ID = :assortmentSessionId"
                + " AND C.WHSE_ITM_ID = I.WHSE_ITM_ID AND "
                + "C.SSON_ID = I.SSON_ID order BY C.CONSM_UNT_ID, C.WHSE_ITM_ID";
        List<ConsmUntItmCustomDto> ConsmUntRes = null;
        try {
                MapSqlParameterSource parameters = new MapSqlParameterSource();
                parameters.addValue("assortmentSessionId", assortmentSessionId);
                NamedParameterJdbcTemplate template = new NamedParameterJdbcTemplate(
                        this.primaryJdbcTemplate.getDataSource());
                ConsmUntRes = template.query(query, parameters, this.rmConsmUntItm);
        }
        catch (DataAccessException e) {}
        return ConsmUntRes;
}

@Override
private final RowMapper<ConsmUntItmCustomDto> rmConsmUntItm = new RowMapper<ConsmUntItmCustomDto>()
{
        @Override
        public ConsmUntItmCustomDto mapRow(ResultSet rs, int rowNum) throws SQLException {
                ConsmUntItmCustomDto dto = new ConsmUntItmCustomDto();
                dto.setConsmUntId(rs.getLong("CONSM_UNT_ID"));
                dto.setLgcyId(rs.getInt("LGCY_ID"));
                dto.setWhseItmId(rs.getInt("WHSE_ITM_ID"));
                return dto;
        }
};
```

Spring Data

@Entity
@Table(name="")
@Id
@Column
@Transient - Specifies that the property or field is not persistent.
@OneToOne(fetch = FetchType.EAGER, cascade = {CascadeType.PERSIST, CascadeType.MERGE, CascadeType.REMOVE})

Spring Security

- module of the Spring framework that provides authentication and authorization methods in Java applications

Authentication – who is this user? Verify if user
Authorization – is user allowed to do this
Principal – currently logged in user
Granted Authority –
Roles – group of authorities

spring-boot-starter-security – dependency

- Adds mandatory authentication for URL
- Adds login form,
- Handles login error
- Creates user name and password

For user in application.properties

Spring.security.user.name=
Spring.security.user.password=

Create a class with @EnableWebSecurity annotation and extends the class to WebSecurityConfigurerAdapter

And implements it's 2 methods configure(AuthenticationManagerBuilder auth) and configure(HttpSecurity http)

Connection to database instead of using the user entity and UserdDetailsService

- Create a user model that extends UserDetails
- Create a Service that extends UserDetailService

LDAP

LDAP – lightweight directory access protocol, use to store and maintain organizational directory information over IP network. Directory of user stored in LDAP Server

<>spring-security-ldap</>

Spring has out of the box class which know how to connect to LDAP. You just need to configure

JWT

Tokens are used to identify who is requesting, so the user doesn't need to provide their identity again for subsequent requests,

Authorization Stategies

1. Session Token - reference token that contains id of user stored in cookies and compare it in session logs.
   - use for a single server that runs the application.
   - use for multiple servers that runs the application thru load balancer.
2. JWT - Json Web Tokens – uses token that its encrypted and stored in header which allows communication and request between server and user, without the need to remember the identity.

   //Structure. Header, Payload (value that is being sent), Signature

Spring AOP

Aspect-oriented programming - programming technique which allows programmers to modularize crosscutting concerns.

> **Concern** – is behavior which we want to have in a module of an application.
>
> **Cross-cutting concern** – It's a concern which is applicable throughout the application and it affects the entire application. e.g. logging , security.

Aspect is a modularization of concern which cuts across multiple objects. a class that implements cross-cutting concerns, such as transaction management

Advice: the methods that get executed when a specific JoinPoint with matching Pointcut is reached in the application. An Action taken by an aspect at a particular joinpoint. This is the actual action to be taken either before or after the method execution.

JoinPoint: A point during the execution of a program and represents a method execution.

Pointcut: a set of regular expressions that are matched with JoinPoint to determine whether Advice needs to be executed or not.

Proxy

-An object which is created after applying advice to a target object is known as a Proxy.
-Advice + Target Object = Proxy

Different types of Advices in Spring AOP are:

a. **Before:** These types of advices execute before the joinpoint methods and are configured using **@Before** annotation mark.
b. **After returning:** These types of advices execute after the joinpoint methods completes executing normally and are configured using @AfterReturning annotation mark.
c. **After throwing:** These types of advices execute only if joinpoint method exits by throwing an exception and are configured using @AfterThrowing annotation mark.
d. **After (finally):** These types of advices execute after a joinpoint method, regardless of the method's exit whether normally or exceptional return and are configured using @After annotation mark.
e. **Around:** These types of advices execute before and after a joinpoint and are configured using @Around annotation mark.

different AOP implementations?

> Spring Aop, Apache AspectJ, JBoss Aop.

Weaving- process of linking an aspect with other application types or objects to create an advised object.